



PUC-SP



MCP Smart Incident Analyzer

CO Distributed communication for AI using MCP (Model Context Protocol)

Professor: Carlos Eduardo Paes

Autores: Fabiana Campanari, Pedro Vycor Almeida

TABLE OF CONTENTS

- Visão geral executiva
- Executive Overview
- Problem Context
- Project Objectives
- Conceptual Foundations and Use of
- Current Implementation and Project Structure
- Technical Analysis Findings
- Performance, Testing, and Quality
- Risks, Limitations, and Improvements
- Executive Conclusion

EXECUTIVE OVERVIEW

The MCP Smart Incident Analyzer is a distributed architecture proposal for intelligent analysis of incidents in AI systems, designed to transform complex occurrences into traceable, explainable, documentable, and auditable cases. The solution adopts MCP as the conceptual foundation for structured communication between components, agents, and specialized services, with a focus on modularity, interoperability, and analytical clarity.

The project also positions itself as both an academic and technical artifact, functioning as documentation, report foundation, presentation support, and a demonstration of architecture applied to AI incident analysis. The complementary technical analysis shows that the current implementation already presents a modular architecture, validated tests, extensive documentation, and a solid foundation for future expansion.

PROBLEM CONTEXT

The growing use of intelligent systems in corporate, academic, and operational environments has also led to an increase in failures, inadequate responses, biases, unexpected behaviors, security incidents, and regulatory risks. In such scenarios, incidents do not arise from a single point in the system, but from the interaction between data input, provided context, activated models, consulted tools, intermediate decisions, and final responses.

This makes incident analysis in AI inherently a distributed, contextual, and multifactorial problem. Monolithic or low-observability systems make it difficult to reconstruct what happened, identify the root cause, and produce auditable explanations, which justifies the adoption of a distributed and context-oriented architecture.

PROJECT OBJECTIVES

The main objective of the project is to develop a platform capable of analyzing incidents in AI systems in a structured, explainable, extensible, and scalable way, using MCP as the communication foundation between distributed components. Specific objectives include standardizing context exchange between modules, improving incident traceability, organizing communication between specialized services, and creating a solid base for future expansion with new MCP servers.

The proposal also aims to structure a project that simultaneously serves as a technical implementation, academic documentation, architectural demonstration, and presentation support. This hybrid nature is reinforced by the technical analysis results, which indicate extensive documentation relative to the codebase and high levels of readability and organization.

CONCEPTUAL FOUNDATIONS AND USE OF MCP

MCP is presented in the project as a protocol designed for structured context exchange between models, applications, agents, and tools, enabling the organization of how relevant information flows across AI components in a standardized way. Within the solution, the use of this protocol helps transport context between system parts, structure requests and responses, facilitate interoperability between services, reduce module coupling, and enable clearer analytical orchestration.

This architectural decision is significant because AI incidents can rarely be reliably analyzed without sufficient context. Therefore, the solution treats context as a central element of technical investigation, connecting input, classification, history, and analytical consolidation into a coherent flow.

SOLUTION ARCHITECTURE

The proposed architecture was designed to emphasize five core characteristics: modularity, structured communication, scalability, traceability, and extensibility. The flow starts from the user and input interface, passes through the incident orchestrator and MCP client, distributes context to specialized servers, and returns to the analytical layer for consolidation before final presentation.

The technical analysis confirms that the current implementation follows a distributed client-server pattern with three layers: presentation, application, and analysis. In this structure, the MCP client acts as the user interface, the MCP server orchestrates requests, and the incident analyzer serves as the analysis engine responsible for classification, prioritization, and recommendation generation.

AND OPERATIONAL FLOW

The main components are: input interface, incident orchestrator, MCP client, specialized MCP servers, analytical layer, and output layer. Each element performs a specific role in the incident processing cycle, reducing coupling and improving modular evolution.

The operational flow follows an organized sequence of eight steps: incident registration or query, input reception and validation, case structuring, context distribution via MCP, execution by specialized servers, return of results to the orchestrator, analytical consolidation, and presentation of output in a comprehensible format.

CURRENT IMPLEMENTATION AND PROJECT STRUCTURE

In the analyzed implementation, the system core is concentrated in three components: server, client, and tests. The technical report indicates approximately 800 lines of Python

code distributed across seven files, with three main modules, three core classes, and three API-exposed methods.

The code distribution reflects clear separation of responsibilities. The server manages asynchronous TCP connections, validates JSON-RPC 2.0, and routes methods. The client establishes connections, builds requests, and presents results. The testing layer covers connectivity and functional flow.

TECHNICAL ANALYSIS FINDINGS

The technical analysis concludes that the solution presents a solid, modular, and extensible architecture, with clean code, robust documentation, and a well-implemented standardized protocol. Key results include validated tests, adherence to JSON-RPC 2.0, extensive documentation, and consistent performance in controlled environments.

The report also identifies positive internal quality indicators such as low cyclomatic complexity, concise functions, zero code duplication, full documentation coverage, and strong use of type hints. In terms of compliance, it shows full adherence to coding standards and absence of significant technical debt.

PERFORMANCE, TESTING, AND QUALITY

Benchmarks indicate average response times between 50 ms and 100 ms, low communication latency, and efficient incident processing. In terms of capacity, the solution demonstrates high throughput and potential for large-scale processing in controlled environments.

The testing suite stands out for its high coverage and reliability, with all core scenarios validated, including connectivity, valid method execution, and error handling.

RISKS, LIMITATIONS, AND IMPROVEMENTS

Despite positive results, important limitations related to security and operational maturity were identified. Key risks include lack of authentication, absence of encryption in connections, and no rate limiting.

Recommendations include implementing authentication and authorization mechanisms, adopting secure communication protocols, integrating a database for persistence, enabling observability through structured logs and metrics, developing an analytical web interface, and expanding tests to include load, stress, and regression scenarios. Future enhancements include machine learning-based classification, anomaly detection, horizontal scalability, and integration with enterprise tools.

EXECUTIVE CONCLUSION

The MCP Smart Incident Analyzer demonstrates both technical and academic viability as a distributed architecture solution for analyzing incidents in AI systems. Its main strength lies in combining a coherent conceptual proposal with a functional, tested, and well-documented implementation.

The project delivers value across three dimensions: as a technical experiment, as a structured academic artifact, and as a foundation for future evolution. It is a promising solution with strong potential for expansion in security, persistence, analytical interfaces, and advanced intelligence applied to incident analysis.